

MATHDNN_HW5

October 18, 2021

Problem 1

```
[1]: import torch
    from torch import nn

[2]: def sigma(x):
    return torch.sigmoid(x)
    def sigma_prime(x):
    return sigma(x)*(1-sigma(x))

[3]: torch.manual_seed(0)
    L = 6
    X_data = torch.rand(4, 1)
    Y_data = torch.rand(1, 1)

[4]: A_list, b_list = [], []
    for _ in range(L-1):
        A_list.append(torch.rand(4, 4))
        b_list.append(torch.rand(4, 1))
    A_list.append(torch.rand(1, 4))
    b_list.append(torch.rand(1, 1))

[5]: '''
    # Option 1: directly use PyTorch's autograd feature
    for A in A_list:
        A.requires_grad = True
    for b in b_list:
        b.requires_grad = True

    y = X_data
    for ell in range(L):
        S = sigma if ell<L-1 else lambda x: x
        y = S(A_list[ell]@y+b_list[ell])

    # backward pass in pytorch
    loss=torch.square(y-Y_data)/2
    loss.backward()
```

```

print(A_list[0].grad)
'''

'''

# Option 2: construct a NN model and use backprop
class MLP(nn.Module) :
    def __init__(self) :
        super().__init__()
        self.linear = nn.ModuleList([nn.Linear(4,4) for _ in range(L-1)])
        self.linear.append(nn.Linear(4,1))
        for ell in range(L):
            self.linear[ell].weight.data = A_list[ell]
            self.linear[ell].bias.data = b_list[ell].squeeze()

    def forward(self, x) :
        x = x.squeeze()
        for ell in range(L-1):
            x = sigma(self.linear[ell](x))
        x = self.linear[-1](x)
        return x

model = MLP()

loss = torch.square(model(X_data)-Y_data)/2
loss.backward()

print(model.linear[0].weight.grad)
'''

# Option 3: implement backprop yourself
y_list = [X_data]
y = X_data
for ell in range(L):
    S = sigma if ell<L-1 else lambda x: x
    y = S(A_list[ell]@y+b_list[ell])
    y_list.append(y)

dA_list = []
db_list = []
dy = y-Y_data # dloss/dy_L
for ell in reversed(range(L)):
    S = sigma_prime if ell<L-1 else lambda x: torch.ones(x.shape)
    A, b, y = A_list[ell], b_list[ell], y_list[ell]
    v = S(torch.reshape(A@y+b,(-1,)))

    db = dy@torch.diag(v) # dloss/db_ell

```

```

dA = torch.diag(v)@torch.t(dy)@torch.t(y) # dloss/dA_ell
dy = dy@torch.diag(v)@A # dloss/dy_{ell-1}, for the next iteration

dA_list.insert(0, dA)
dB_list.insert(0, dB)

print(dA_list[0])

```

```

tensor([[2.3943e-05, 3.7064e-05, 4.2687e-06, 6.3700e-06],
        [3.4104e-05, 5.2794e-05, 6.0804e-06, 9.0735e-06],
        [2.4438e-05, 3.7831e-05, 4.3571e-06, 6.5019e-06],
        [2.0187e-05, 3.1250e-05, 3.5991e-06, 5.3707e-06]])

```

Problem 6

```

[6]: import torch
import torch.nn as nn

```

```

[7]: # Make sure to use only 10% of the available MNIST data.
# Otherwise, experiment will take quite long (around 90 minutes).

```

```

# (Modified version of AlexNet)
class AlexNet(nn.Module):
    def __init__(self, num_class=10):
        super(AlexNet, self).__init__()

        self.conv_layer1 = nn.Sequential(
            nn.Conv2d(1, 96, kernel_size=4),
            nn.ReLU(inplace=True),
            nn.Conv2d(96, 96, kernel_size=3),
            nn.ReLU(inplace=True)
        )

        self.conv_layer2 = nn.Sequential(
            nn.Conv2d(96, 256, kernel_size=5, padding=2),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2)
        )

        self.conv_layer3 = nn.Sequential(
            nn.Conv2d(256, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 384, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(384, 256, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.MaxPool2d(kernel_size=3, stride=2)
        )

        self.fc_layer1 = nn.Sequential(

```

```

        nn.Dropout(),
        nn.Linear(6400, 800),
        nn.ReLU(inplace=True),
        nn.Linear(800, 10)
    )

    def forward(self, x):
        output = self.conv_layer1(x)
        output = self.conv_layer2(output)
        output = self.conv_layer3(output)
        output = torch.flatten(output, 1)
        output = self.fc_layer1(output)
        return output

```

```

[8]: from torchvision import datasets
    from torchvision.transforms import transforms
    import time
    import matplotlib.pyplot as plt

```

```

[9]: learning_rate = 0.1
    batch_size = 64
    epochs = 150

```

```

[10]: device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model = AlexNet().to(device)
    loss_function = torch.nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

    train_dataset = datasets.MNIST(root='./mnist_data/',
                                   train=True,
                                   transform=transforms.ToTensor(),
                                   download=True)

    train_dataset.data = train_dataset.data[:len(train_dataset.data) // 10]
    train_dataset.targets = train_dataset.targets[:len(train_dataset.targets) // 10]
    train_dataset.targets = torch.randint(0, 9, train_dataset.targets.shape)

    train_loader = torch.utils.data.DataLoader(dataset=train_dataset,
        ↪batch_size=100, shuffle=True)

```

Downloading <http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz>
 Downloading <http://yann.lecun.com/exdb/mnist/train-images-idx3-ubyte.gz> to
 ./mnist_data/MNIST/raw/train-images-idx3-ubyte.gz

0%| | 0/9912422 [00:00<?, ?it/s]

Extracting ./mnist_data/MNIST/raw/train-images-idx3-ubyte.gz to
 ./mnist_data/MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz>

Downloading <http://yann.lecun.com/exdb/mnist/train-labels-idx1-ubyte.gz> to
./mnist_data/MNIST/raw/train-labels-idx1-ubyte.gz

0%| | 0/28881 [00:00<?, ?it/s]

Extracting ./mnist_data/MNIST/raw/train-labels-idx1-ubyte.gz to
./mnist_data/MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz>

Downloading <http://yann.lecun.com/exdb/mnist/t10k-images-idx3-ubyte.gz> to
./mnist_data/MNIST/raw/t10k-images-idx3-ubyte.gz

0%| | 0/1648877 [00:00<?, ?it/s]

Extracting ./mnist_data/MNIST/raw/t10k-images-idx3-ubyte.gz to
./mnist_data/MNIST/raw

Downloading <http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz>

Downloading <http://yann.lecun.com/exdb/mnist/t10k-labels-idx1-ubyte.gz> to
./mnist_data/MNIST/raw/t10k-labels-idx1-ubyte.gz

0%| | 0/4542 [00:00<?, ?it/s]

Extracting ./mnist_data/MNIST/raw/t10k-labels-idx1-ubyte.gz to
./mnist_data/MNIST/raw

/usr/local/lib/python3.7/dist-packages/torchvision/datasets/mnist.py:498:
UserWarning: The given NumPy array is not writeable, and PyTorch does not
support non-writeable tensors. This means you can write to the underlying
(supposedly non-writeable) NumPy array using the tensor. You may want to copy
the array to protect its data or make it writeable before converting it to a
tensor. This type of warning will be suppressed for the rest of this program.
(Triggered internally at /pytorch/torch/csrc/utils/tensor_numpy.cpp:180.)
return torch.from_numpy(parsed.astype(m[2], copy=False)).view(*s)

```
[11]: tick = time.time()
loss_list = []
acc_list = []
for epoch in range(epochs):
    print(f"\nEpoch {epoch + 1} / {epochs}")
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        output = model(images)
        loss = loss_function(output, labels)
        loss.backward()
        optimizer.step()

    loss_list.append(loss)
```

```
pred = output.max(1, keepdim=True)[1]
acc_list.append(pred.eq(labels.view_as(pred)).sum().item() / labels.size(0))
```

Epoch 1 / 150

/usr/local/lib/python3.7/dist-packages/torch/nn/functional.py:718: UserWarning: Named tensors and all their associated APIs are an experimental feature and subject to change. Please do not use them for anything important until they are released as stable. (Triggered internally at

/pytorch/c10/core/TensorImpl.h:1156.)

return torch.max_pool2d(input, kernel_size, stride, padding, dilation, ceil_mode)

Epoch 2 / 150

Epoch 3 / 150

Epoch 4 / 150

Epoch 5 / 150

Epoch 6 / 150

Epoch 7 / 150

Epoch 8 / 150

Epoch 9 / 150

Epoch 10 / 150

Epoch 11 / 150

Epoch 12 / 150

Epoch 13 / 150

Epoch 14 / 150

Epoch 15 / 150

Epoch 16 / 150

Epoch 17 / 150

Epoch 18 / 150

Epoch 19 / 150

Epoch 20 / 150

Epoch 21 / 150

Epoch 22 / 150

Epoch 23 / 150

Epoch 24 / 150

Epoch 25 / 150

Epoch 26 / 150

Epoch 27 / 150

Epoch 28 / 150

Epoch 29 / 150

Epoch 30 / 150

Epoch 31 / 150

Epoch 32 / 150

Epoch 33 / 150

Epoch 34 / 150

Epoch 35 / 150

Epoch 36 / 150

Epoch 37 / 150

Epoch 38 / 150

Epoch 39 / 150

Epoch 40 / 150

Epoch 41 / 150

Epoch 42 / 150

Epoch 43 / 150

Epoch 44 / 150

Epoch 45 / 150

Epoch 46 / 150

Epoch 47 / 150

Epoch 48 / 150

Epoch 49 / 150

Epoch 50 / 150

Epoch 51 / 150

Epoch 52 / 150

Epoch 53 / 150

Epoch 54 / 150

Epoch 55 / 150

Epoch 56 / 150

Epoch 57 / 150

Epoch 58 / 150

Epoch 59 / 150

Epoch 60 / 150

Epoch 61 / 150

Epoch 62 / 150

Epoch 63 / 150

Epoch 64 / 150

Epoch 65 / 150

Epoch 66 / 150

Epoch 67 / 150

Epoch 68 / 150

Epoch 69 / 150

Epoch 70 / 150

Epoch 71 / 150

Epoch 72 / 150

Epoch 73 / 150

Epoch 74 / 150

Epoch 75 / 150

Epoch 76 / 150

Epoch 77 / 150

Epoch 78 / 150

Epoch 79 / 150

Epoch 80 / 150

Epoch 81 / 150

Epoch 82 / 150

Epoch 83 / 150

Epoch 84 / 150

Epoch 85 / 150

Epoch 86 / 150

Epoch 87 / 150

Epoch 88 / 150

Epoch 89 / 150

Epoch 90 / 150

Epoch 91 / 150
Epoch 92 / 150
Epoch 93 / 150
Epoch 94 / 150
Epoch 95 / 150
Epoch 96 / 150
Epoch 97 / 150
Epoch 98 / 150
Epoch 99 / 150
Epoch 100 / 150
Epoch 101 / 150
Epoch 102 / 150
Epoch 103 / 150
Epoch 104 / 150
Epoch 105 / 150
Epoch 106 / 150
Epoch 107 / 150
Epoch 108 / 150
Epoch 109 / 150
Epoch 110 / 150
Epoch 111 / 150
Epoch 112 / 150
Epoch 113 / 150
Epoch 114 / 150

Epoch 115 / 150

Epoch 116 / 150

Epoch 117 / 150

Epoch 118 / 150

Epoch 119 / 150

Epoch 120 / 150

Epoch 121 / 150

Epoch 122 / 150

Epoch 123 / 150

Epoch 124 / 150

Epoch 125 / 150

Epoch 126 / 150

Epoch 127 / 150

Epoch 128 / 150

Epoch 129 / 150

Epoch 130 / 150

Epoch 131 / 150

Epoch 132 / 150

Epoch 133 / 150

Epoch 134 / 150

Epoch 135 / 150

Epoch 136 / 150

Epoch 137 / 150

Epoch 138 / 150

Epoch 139 / 150

Epoch 140 / 150

Epoch 141 / 150

Epoch 142 / 150

Epoch 143 / 150

Epoch 144 / 150

Epoch 145 / 150

Epoch 146 / 150

Epoch 147 / 150

Epoch 148 / 150

Epoch 149 / 150

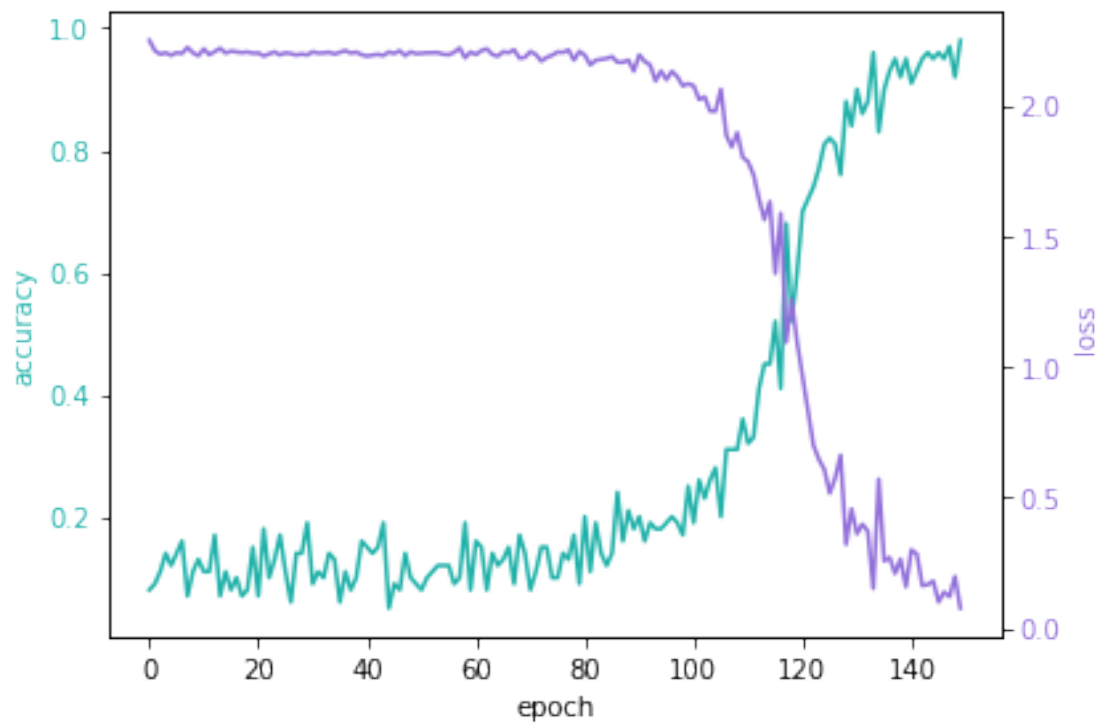
Epoch 150 / 150

```
[12]: fig, ax1 = plt.subplots()
      ax1.set_xlabel('epoch')
      ax1.set_ylabel('accuracy', color='lightseagreen')
      l1, = ax1.plot(range(len(acc_list)), acc_list, color='lightseagreen')
      ax1.tick_params(axis='y', labelcolor='lightseagreen')

      ax2 = ax1.twinx()
      ax2.set_ylabel('loss', color='mediumpurple')
      l2, = ax2.plot(range(len(loss_list)), loss_list, color='mediumpurple')
      ax2.tick_params(axis='y', labelcolor='mediumpurple')

      fig.tight_layout()
      plt.show()

      tock = time.time()
      print(f"Total training time: {tock - tick}")
```



Total training time: 1725.6406500339508